EVIDENCERANK

Evidence-Aware Vertical Search for Scientific Literature

CSD358 — Information Retrieval Hackathon / Midsem Project

Team Member	Student ID	Primary Contribution
S. Sai Kumar	2410110278	System design, corpus preparation, EvidenceRank ranking, desktop interface, integration/demo
Abhinay	2410110015	Retrieval modules, relevance judgments, evaluation, baseline comparison, experimental analysis

Executive Summary

EvidenceRank is a local desktop vertical-search prototype for scientific literature. It addresses a specific weakness of ordinary keyword search: in a scientific setting, two papers with similar textual relevance may differ substantially in evidence value because one is more cited or more recent. The system therefore separates textual relevance from evidence signals and re-ranks the retrieved literature using relevance, citation authority and recency.

The prototype uses a focused corpus of 83 unique scientific papers collected from OpenAlex across four health/environment topics. The IR pipeline includes normalization, an inverted index, TF-IDF vector-space retrieval with cosine similarity, BM25 as an additional lexical retriever, title/abstract zone search, and top-K ranking. The core evaluated EvidenceRank score is a transparent weighted combination of normalized textual relevance, citation authority and recency.

On a small self-judged evaluation set of 5 queries and 25 binary judgments, EvidenceRank records Mean P@5 = 0.800 and Mean Recall@5 = 0.853, compared with stored TF-IDF baseline values of 0.760 and 0.813. The observed improvement is +0.040 on both metrics. These results are preliminary and are not presented as generalizable evidence beyond the judged prototype set.

Project at a Glance

Dimension	Implemented choice
Track	T6 — Vertical Search for Science
Corpus	83 unique OpenAlex scientific works; title, abstract, year, citation count and source metadata
Core IR	Preprocessing, inverted index, TF-IDF, cosine similarity, BM25, zone weighting, top-K ranking
Evidence signals	Citation authority via log(1 + citations); recency via inverse publication age
Interface	PySide6 desktop application; runs locally with python main.py
Evaluation	5 judged queries; P@5 and Recall@5; TF-IDF baseline comparison

Central idea: scientific search should rank not only what matches the query, but also what is stronger evidence for the researcher to inspect first.

1. Problem, User Need and Track Relevance

1.1 Problem Definition

Scientific search has a domain-specific requirement: users are usually not looking for any document containing the right words; they need a small set of papers that are both topically relevant and useful as evidence. A pure lexical ranking can over-reward a paper that matches the query frequently while ignoring useful metadata such as publication year and citation history.

EvidenceRank frames the task as vertical search. The document type is scientific papers rather than arbitrary web pages, and the ranking model uses scientific metadata as a static-quality signal. The user enters a research query, the system retrieves candidates, computes transparent component scores, and presents the top results with the evidence signals visible.

1.2 Why T6 — Vertical Search for Science

The chosen track is T6, whose research theme is vertical search for specialized domains such as law, finance or science. The project directly fits the science case because the corpus consists of scholarly works and the ranking includes scientific metadata, especially citation counts, publication year, title and abstract. This is not a generic web-search wrapper: the ranking objective is deliberately adapted to a scientific information need.

1.3 User Need and Design Objectives

User need	Design response	Why it matters
Find relevant scientific papers quickly	TF-IDF + cosine similarity over title and abstract	Transparent lexical relevance foundation
Prioritize stronger evidence	Citation authority signal	Separates textual match from evidence/impact metadata
Avoid only old highly cited papers	Recency signal	Balances authority with freshness
Exploit scientific structure	Title/abstract zone search	Title matches can be stronger topical signals
Inspect why a result ranked highly	Component scores in UI	Makes ranking auditable

1.4 Corpus Construction and Data Provenance

The corpus was collected programmatically from the public OpenAlex scholarly metadata service using four focused queries: air pollution health; climate change human health; respiratory disease environment; and cardiovascular disease air pollution. Up to 30 works were requested per query, duplicates were removed by OpenAlex work ID, and the resulting prototype corpus contains 83 unique papers. Stored fields include title, abstract, publication year, citation count, source and landing URL.

This corpus is intentionally a focused prototype rather than a claim of web-scale coverage. It is large enough to make ranking choices meaningful for the demonstration while remaining inspectable during a short hackathon build. A production version would require a much larger and continuously refreshed scholarly index.

1.5 Scope

- **In scope:** scientific paper retrieval, evidence-aware re-ranking, transparent scoring, local desktop search, and quantitative sanity-check evaluation.
- **Out of scope:** full scholarly crawling, citation-graph inference, learning-to-rank from large relevance datasets, semantic embeddings, and production-scale deployment.

2. Information Retrieval Methodology

2.1 End-to-End IR Pipeline

Stage	Implementation	IR purpose
1. Collection	collect_corpus.py	Build a domain-specific scientific collection
2. Normalization	preprocessing.py	Lowercase, punctuation removal, tokenization
3. Indexing	index.py	Dictionary/postings-style inverted index
4. Vector retrieval	retrieval.py	TF-IDF vectors + cosine similarity
5. Lexical alternative	bm25.py	BM25 ranking as an outside-syllabus IR method
6. Field-aware retrieval	zone_search.py	Separate title and abstract scores
7. Evidence re-ranking	evidence_rank.py	Combine relevance, citation authority and recency
8. Top-K assembly	ranking stage	Return highest-scoring candidates
9. Evaluation	evaluate.py / comparison.py	P@5/Recall@5 and baseline comparison

2.2 Preprocessing and Inverted Index

The preprocessing function performs case folding, punctuation removal and whitespace tokenization. This keeps lexical matching stable across capitalization and punctuation variants. The inverted index maps each normalized term to paper IDs. Although the final EvidenceRank score uses vector-space similarity, the explicit index demonstrates the classic IR dictionary/postings model and provides a foundation for future Boolean or optimized postings-based retrieval.

2.3 TF-IDF and Cosine Similarity

TF-IDF represents each paper and query as weighted term vectors. Frequent terms receive lower discriminative weight through inverse document frequency, while terms important to a particular paper receive higher term weight. Cosine similarity measures the angle between the query and document vectors, producing the baseline relevance score.

2.4 BM25 and Zone Search

BM25 is implemented as a complementary lexical retrieval method using tokenized title+abstract text. It is useful because its scoring behavior differs from TF-IDF and is a standard ranking function outside the core lecture set. Zone search separately scores title and abstract fields and uses 0.70 × title + 0.30 × abstract, reflecting the intuition that a direct title match is a strong signal of research intent.

2.5 Why These IR Choices

Choice	Reason
TF-IDF + cosine	Simple, interpretable, strong baseline and directly aligned with vector-space IR lectures.
Inverted index	Makes the system recognizably an IR engine rather than a generic text-processing script.
BM25	Adds a standard lexical ranking alternative and demonstrates extension beyond the lecture baseline.
Zone weighting	Uses document structure, especially meaningful for scientific titles and abstracts.
Top-K ranking	Matches the real workflow: researchers inspect a small ranked shortlist.

3. EvidenceRank: Evidence-Aware Ranking and Novelty

3.1 Core Ranking Model

The central contribution is a transparent re-ranking layer. Instead of replacing conventional retrieval, EvidenceRank first obtains a textual relevance estimate and then incorporates domain-specific evidence signals. Each component is normalized before fusion so that one raw scale does not dominate another.

EvidenceRank(d,q) = 0.60 × Relevance(d,q) + 0.20 × Authority(d) + 0.20 × Recency(d)

Component	Computation	Interpretation
Relevance	Normalized cosine similarity between query and title+abstract TF-IDF vector	How well the paper matches the query
Authority	Normalize(log(1 + citation_count))	Citation-based scholarly attention
Recency	Normalize(1 / (1 + current_year − publication_year))	Rewards newer papers without dominating
Final score	0.60 / 0.20 / 0.20 weighted fusion	Balances match, authority and freshness

3.2 Why the Log Transform?

Citation counts are highly skewed: a small number of papers can have much larger citation counts than most of the corpus. Using log(1+c) compresses this long tail before min-max normalization. This prevents a very highly cited

3.3 Novelty / Proposed Contribution

The project's novelty is a transparent evidence-aware re-ranking layer designed specifically for scientific vertical search. Unlike the TF-IDF baseline, which ranks papers mainly by textual similarity, EvidenceRank treats citation authority and publication recency as explicit ranking signals alongside relevance. This produces a single, interpretable ranking score while showing the contribution of each

Baseline limitation → proposed contribution:
TF-IDF answers "Which papers match my words?" EvidenceRank extends this with citation authority and recency, making scientific evidence quality and freshness explicit, visible and interpretable in the

3.4 Supporting Experimental Search Engine

The repository also contains search_engine.py, an experimental fusion module that combines TF-IDF, BM25, title/abstract zones, phrase matching, citation authority, recency and metadata filters. This demonstrates how the prototype can be extended toward a richer hybrid ranker. For methodological clarity, the reported evaluation numbers correspond to the simpler core EvidenceRank implementation used by the evaluated ranker and evaluation script.

3.5 Transparency

- Every result exposes final score plus relevance, authority and recency components.
- The ranking formula is deterministic and inspectable; no opaque learned model is required.
- The user can inspect paper metadata and the system remains fully local during demonstration.

4. Working System and End-to-End Implementation

4.1 System Architecture

User query	→	Preprocess / vectorize	→	Retrieve candidates	→	EvidenceRank	→	Top-K UI
Scientific text	→	TF-IDF / BM25 / zones	→	Relevance scores	→	Authority + recency	→	Ranked papers

4.2 Code-to-Function Mapping

File	Responsibility	Evidence shown in demo
collect_corpus.py	OpenAlex collection and deduplication	Corpus provenance and 83-paper collection
preprocessing.py	Case folding, punctuation removal, tokenization	Preprocessing concept
index.py	Inverted index / postings-style mapping	IR indexing output
retrieval.py	TF-IDF + cosine baseline	Baseline retrieval scores
bm25.py	BM25 lexical retrieval	Alternative IR ranking
zone_search.py	Title/abstract field weighting	Structured scientific retrieval
evidence_rank.py	Core EvidenceRank formula	Final/authority/recency scores
evaluate.py	P@5 and Recall@5 computation	Quantitative evaluation
comparison.py	Formatted baseline comparison	Comparison table
app.py / main.py	PySide6 interface and entry point	Live end-to-end search

4.3 Live Workflow

Run python main.py. Enter a real scientific query such as “air pollution cardiovascular disease”. The application ranks the corpus and displays the top five papers. Each card exposes the EvidenceRank score, relevance, authority, recency, year and citation count. “View details” opens the paper metadata.

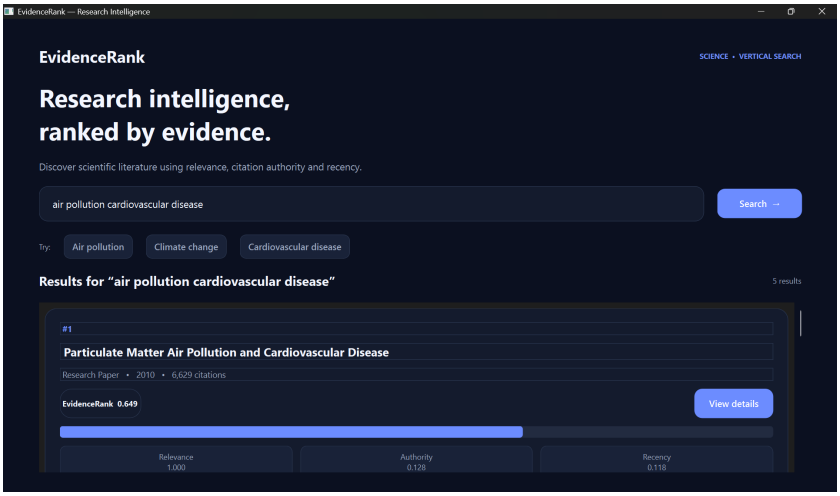


Figure 1. EvidenceRank desktop search interface running locally. The UI is a presentation layer; the graded contribution is the underlying IR pipeline and evaluation.

4.4 Working-System Requirement

The system is a genuine local implementation rather than a screenshot-only mock. The repository contains executable Python modules, a local corpus, an evaluation script and a desktop entry point. The assignment explicitly permits local execution and does not require online deployment.

5. Evaluation Methodology

5.1 Evaluation Design

The evaluation is a small, self-judged sanity-check experiment designed to test whether evidence-aware ranking changes top-K quality relative to a text-only baseline. Five research queries were used, and five top-ranked candidate papers were judged for each query, producing 25 binary relevance judgments. The evaluation is intentionally modest because this is a hackathon prototype; it should not be interpreted as a statistically comprehensive scholarly-search benchmark.

#	Query	Judged candidates
1	air pollution cardiovascular disease	5
2	air pollution respiratory disease	5
3	climate change human health	5
4	particulate pollution health	5
5	air pollution mortality	5
Total	5 queries	25 judgments

5.2 Metrics

Precision@5 (P@5) measures the fraction of the five returned papers that are judged relevant. Recall@5 measures the fraction of judged relevant items recovered within the top five, relative to the relevant set available in the evaluation setup.

P@5 = relevant documents in top 5 / 5
Recall@5 = relevant documents in top 5 / total judged relevant documents

5.3 Baseline

The baseline is conventional TF-IDF + cosine similarity over title and abstract text. It is the appropriate comparison because it isolates the effect of adding scientific evidence signals: the baseline asks how well text relevance alone performs, while EvidenceRank adds citation authority and recency to the ranking objective.

5.4 Reproducibility Path

Step	Command / artifact	Purpose
Run retrieval	python main.py	Live search on real corpus input
Run evaluation	python src\evaluate.py	Compute P@5 and Recall@5
Show comparison	python src\comparison.py	Display stored TF-IDF vs EvidenceRank metrics
Judgment data	data/raw/judgments.json	Query-level document IDs and relevance labels

5.5 Evaluation Integrity and Caveat

The evaluation set is intentionally small and the relevance labels are self-judged. In addition, the comparison script is a reporting script that prints stored baseline measurements; the actual EvidenceRank metric computation is performed in evaluate.py. These choices are disclosed rather than hidden. A stronger evaluation would use more queries, independent assessors, graded relevance, and a larger scientific benchmark.

6. Results, Interpretation and Future Work

6.1 Quantitative Results

Metric	TF-IDF baseline	EvidenceRank	Absolute improvement
Mean P@5	0.760	0.800	+0.040
Mean Recall@5	0.813	0.853	+0.040

The results show a consistent positive change on both reported metrics. EvidenceRank improves the precision of the top five results and also recovers a larger fraction of the judged relevant set. The important interpretation is not that citation and recency are universally superior, but that adding domain-specific evidence signals can improve ranking quality on the prototype's judged queries.

6.2 Ranking Design Trade-offs

Design choice	Decision	IR rationale
Relevance weight	60%	Keeps query-document match as the primary signal; evidence metadata should not overwhelm topical relevance.
Authority weight	20%	Adds citation-based static quality while remaining secondary to lexical relevance.
Recency weight	20%	Balances freshness with older foundational work; avoids making publication year the dominant criterion.

6.3 Practical Failure Cases and Robustness

A high citation count does not guarantee scientific correctness, and a recent paper is not automatically better than a foundational study. The model therefore uses authority and recency only as secondary signals. Normalization also prevents raw citation magnitude or publication-year scale from dominating the relevance component.

6.4 Limitations

- The corpus contains 83 papers, so it is a focused prototype rather than a broad scholarly index.
- Only five queries and 25 binary judgments are used; statistical confidence is limited.
- Citation count is a proxy for authority, not a direct measure of scientific correctness.
- Recency can disadvantage foundational older work if weighted too strongly.
- The system does not yet use citation-graph structure, semantic embeddings, learned ranking, or independent expert judgments.

6.5 Next Steps

A continuation should expand the corpus, add independent relevance assessors, evaluate graded relevance and NDCG@k, incorporate citation-graph features, add semantic retrieval as a complementary candidate generator, and learn ranking weights from judged data. The interface can then expose year/citation filters while preserving the transparent evidence breakdown.

7. Work Division, Conclusion and References

7.1 Work Division

Member	Student ID	Contribution
S. Sai Kumar	2410110278	System design; scientific corpus preparation and provenance; EvidenceRank ranking methodology; desktop UI design; main entry-point integration; end-to-end demonstration.
Abhinay	2410110015	Retrieval-method implementation; TF-IDF/BM25/zone-search analysis; relevance-judgment setup; evaluation metrics; baseline comparison; experimental interpretation and evaluation demonstration.

7.2 Reproducibility and Responsible Development

The project is designed to be reproducible locally: the corpus is stored in data/raw/papers.json, relevance labels are stored in data/raw/judgments.json, and the main entry point is python main.py. Evaluation can be reproduced with python src\evaluate.py, while python src\comparison.py displays the recorded baseline comparison.

7.3 Conclusion

EvidenceRank demonstrates how classical Information Retrieval can be adapted to a scientific vertical without abandoning transparency. The system begins with standard lexical retrieval, preserves an inspectable relevance score, and adds two domain-relevant signals, citation authority and recency, to form an evidence-aware ranking. The prototype is runnable locally, uses real scholarly metadata, and is evaluated against a clear TF-IDF baseline. Its measured gains are encouraging within the small judged set, while the disclosed limitations define a concrete path toward a stronger course-project version.

Final takeaway: EvidenceRank does not argue that “more citations = better science.” It argues that scientific search should expose and balance multiple evidence signals instead of treating every textually similar paper as equivalent.

References

[1] CSD358: IR Hackathon — Midsem Assignment 2026. Course brief and evaluation rubric.
[2] Manning, C. D., Raghavan, P., & Schütze, H. Introduction to Information Retrieval. Cambridge University Press.
[3] Robertson, S. & Zaragoza, H. Probabilistic relevance framework: BM25 and related ranking methods.
[4] OpenAlex. Open scholarly metadata service used as the project corpus source. https://openalex.org/
[5] scikit-learn documentation. TF-IDF vectorization and cosine-similarity utilities used in the implementation.
[6] rank_bm25 Python package. BM25 implementation used in the supporting retrieval module.